```
import numpy as np
from typing import Tuple, List

from qiskit.quantum_info import
SparsePauliOp
from qiskit_algorithms import QAOA
from qiskit_algorithms.optimizers import
COBYLA
try:
    from qiskit.primitives import Sampler
except ImportError:
    from qiskit.primitives import
StatevectorSampler as Sampler


# 1. Synthetic Market Data Generation
(unchanged, now with cost vector)
def generate_market_data(seed: int = 42):
    np.random.seed(seed)
    asset_names = ["A1", "A2", "A3", "A4",
"A5", "A6", "A7", "A8", "A9", "A10"]
    mu = np.array([0.08, 0.07, 0.09, 0.03,
0.04, 0.06, 0.02, 0.07, 0.06, 0.01])
    vols = np.array([0.15, 0.16, 0.21,
0.05, 0.07, 0.18, 0.08, 0.14, 0.10, 0.008])
    N = len(asset_names)
    A = np.random.uniform(0.1, 0.7, size=
(N, N))
    corr = (A + A.T) / 2.0
    np.fill_diagonal(corr, 1.0)
```

```
    D = np.diag(vols)
    sigma = D @ corr @ D
    # NEW: synthetic per-asset transaction
cost (bps-scale, roughly tracks
illiquidity/vol)
    cost = np.array([0.010, 0.012, 0.020,
0.004, 0.006, 0.018, 0.009, 0.015, 0.011,
0.001])
    return mu, sigma, cost, asset_names


# 2. QUBO & Ising Mapping Engine (fixed:
cost term added, bit-order fixed)
class QuantumPortfolioMapper:
    def __init__(self, mu, sigma, cost, q:
float = 0.5, gamma: float = 1.0,
                 B: int = 5, P: float =
30.0):
        self.mu, self.sigma, self.cost =
mu, sigma, cost
        self.q, self.gamma, self.B, self.P
= q, gamma, B, P
        self.N = len(mu)
        self.Q_diag, self.Q_off =
self._build_qubo()
        self.ising_op, self.offset =
self._qubo_to_ising()

    def _build_qubo(self):
        # FIX 1: added gamma * cost term
(linear -> only touches Q_diag)
        Q_diag = (self.q *
np.diag(self.sigma) - self.mu + self.gamma
* self.cost
```

```
                  + self.P * (1.0 - 2.0 *
self.B))
        Q_off = 2.0 * self.q * self.sigma +
2.0 * self.P
        np.fill_diagonal(Q_off, 0)
        return Q_diag, Q_off

    def _qubo_to_ising(self):
        pauli_list = []
        for i in range(self.N):
            h_i = -0.5 * self.Q_diag[i] -
0.25 * np.sum(self.Q_off[i, :])
            z_str = ["I"] * self.N
            z_str[i] = "Z"

pauli_list.append(("".join(reversed(z_str))
, h_i))
        for i in range(self.N):
            for j in range(i + 1, self.N):
                z_str = ["I"] * self.N
                z_str[i] = "Z"
                z_str[j] = "Z"

pauli_list.append(("".join(reversed(z_str))
, 0.25 * self.Q_off[i, j]))
        offset = (0.5 * np.sum(self.Q_diag)
+ 0.25 * np.sum(np.triu(self.Q_off, 1))
                  + self.P * (self.B ** 2))
        return
SparsePauliOp.from_list(pauli_list), offset

    def evaluate_bitstring(self, bitstring:
str, already_asset_order: bool = False) ->
float:
```

```
        """
        FIX 2: bit-order correction.
        Qiskit measurement strings are
little-endian: leftmost char = qubit N-1,
        rightmost char = qubit 0. Our
Hamiltonian maps asset i -> qubit i.
        So a raw measured string must be
reversed before it represents
        x = (x_asset0, x_asset1, ...,
x_assetN-1).
        Set already_asset_order=True if
you're passing in a string that is
        already in asset order (e.g. one
you constructed yourself).
        """
        s = bitstring if
already_asset_order else bitstring[::-1]
        x = np.array([int(b) for b in s])
        return (self.q * (x.T @ self.sigma
@ x) - self.mu.T @ x
                + self.gamma * (self.cost @
x)
                + self.P * ((np.sum(x) -
self.B) ** 2))

    def raw_objective(self, x: np.ndarray)
-> float:
        """Objective WITHOUT the budget
penalty term - for reporting true cost."""
        return self.q * (x.T @ self.sigma @
x) - self.mu.T @ x + self.gamma *
(self.cost @ x)
```

```
def to_asset_order(bitstring: str) -> str:
    """Convert a raw Qiskit measurement
string into asset-index order."""
    return bitstring[::-1]


# 3. FIX 3: Feasibility repair - greedily
force Sum(x) == B before local search
def repair_feasibility(x: np.ndarray,
mapper: QuantumPortfolioMapper) ->
np.ndarray:
    x = x.copy()
    B = mapper.B
    current = int(np.sum(x))

    if current > B:
        # remove the most expensive/least
attractive selected assets first
        # marginal contribution of asset i
to raw objective if flipped off
        selected = [i for i in
range(mapper.N) if x[i] == 1]
        # sort by (return - cost) ascending
-> drop worst first
        selected.sort(key=lambda i:
mapper.mu[i] - mapper.gamma *
mapper.cost[i])
        for i in selected:
            if current <= B:
                break
            x[i] = 0
            current -= 1

    elif current < B:
```

```
        unselected = [i for i in
range(mapper.N) if x[i] == 0]
        # add assets with best (return -
cost) first
        unselected.sort(key=lambda i:
mapper.mu[i] - mapper.gamma *
mapper.cost[i], reverse=True)
        for i in unselected:
            if current >= B:
                break
            x[i] = 1
            current += 1

    assert int(np.sum(x)) == B,
"Feasibility repair failed to hit budget
exactly"
    return x


# 4. Hybrid Post-Processing (Classical
Local Search) - now budget-preserving by
construction
def
classical_local_search_refinement(x_init:
np.ndarray, mapper:
QuantumPortfolioMapper):
    current_x = x_init.copy()
    current_cost =
mapper.evaluate_bitstring("".join(map(str,
current_x)), already_asset_order=True)

    improved = True
    while improved:
        improved = False
```

```
        N = len(current_x)
        for i in range(N):
            for j in range(N):
                if current_x[i] == 1 and
current_x[j] == 0:
                    candidate_x =
current_x.copy()
                    candidate_x[i],
candidate_x[j] = 0, 1
                    cand_cost =
mapper.evaluate_bitstring(
                        "".join(map(str,
candidate_x)), already_asset_order=True
                    )
                    if cand_cost <
current_cost:
                        current_cost =
cand_cost
                        current_x =
candidate_x
                        improved = True
                        break
            if improved:
                break

    return current_x, current_cost


# 5. Main Workflow
if __name__ == "__main__":
    mu, sigma, cost, asset_names =
generate_market_data()
    mapper = QuantumPortfolioMapper(mu,
sigma, cost, q=0.5, gamma=1.0, B=5, P=30.0)
```

```
    print(f"Ising Hamiltonian Qubit Count:
{mapper.ising_op.num_qubits}")
    print(f"Energy Shift Offset:
{mapper.offset:.4f}")

    sampler = Sampler()
    optimizer = COBYLA(maxiter=25)
    qaoa = QAOA(sampler=sampler,
optimizer=optimizer, reps=1)
    import time
    _t0 = time.time()

    qaoa_result =
qaoa.compute_minimum_eigenvalue(mapper.isin
g_op)
    print(f"QAOA optimization took
{time.time()-_t0:.1f}s")
    raw_bitstring =
max(qaoa_result.eigenstate,
key=qaoa_result.eigenstate.get)

    # FIX 2 applied: convert measured
(little-endian) string to asset order
    asset_order_str =
to_asset_order(raw_bitstring)
    x_measured = np.array([int(b) for b in
asset_order_str])
    raw_cost =
mapper.evaluate_bitstring(asset_order_str,
already_asset_order=True)

    print(f"\n[QAOA] Raw measured bitstring
(Qiskit order): {raw_bitstring}")
```

```
    print(f"[QAOA] Bitstring in asset order
(x_A1...x_A10): {asset_order_str}")
    print(f"[QAOA] Selected count (Sum x):
{int(np.sum(x_measured))}  (target B=
{mapper.B})")
    print(f"[QAOA] Full objective incl.
penalty: {raw_cost:.4f}")

    # FIX 3: repair feasibility BEFORE
local search, guaranteeing budget is
respected
    x_feasible =
repair_feasibility(x_measured, mapper)
    print(f"\n[Repair] Bitstring after
feasibility repair: {''.join(map(str,
x_feasible))}")
    print(f"[Repair] Selected count (Sum
x): {int(np.sum(x_feasible))}  (target B=
{mapper.B})")

    # Classical local search refinement
(budget-preserving by construction: 1<->0
swaps only)
    x_refined, refined_cost =
classical_local_search_refinement(x_feasibl
e, mapper)
    print(f"\n[Hybrid Refinement] Best
bitstring (asset order): {''.join(map(str,
x_refined))}")
    print(f"[Hybrid Refinement] Selected
count (Sum x): {int(np.sum(x_refined))}
(target B={mapper.B})")
    print(f"[Hybrid Refinement] Full
objective incl. penalty:
```

```
{refined_cost:.4f}")
    print(f"[Hybrid Refinement] Raw
objective (no penalty):
{mapper.raw_objective(x_refined):.4f}")

    selected_indices = [i for i, bit in
enumerate(x_refined) if bit == 1]
    print("\nSelected Asset Portfolio
(B=5):")
    for idx in selected_indices:
        print(f" - {asset_names[idx]}
(Return: {mu[idx]*100:.1f}%, "
              f"Vol: {np.sqrt(sigma[idx,
idx])*100:.1f}%, Cost:
{cost[idx]*100:.2f}%)")

    # --- Sanity check: unit test for bit-
ordering fix ---
    print("\n--- Sanity check: manual x vs
Hamiltonian expectation ---")
    x_test = np.array([1, 1, 1, 1, 1, 0, 0,
0, 0, 0])  # first 5 assets selected
    direct_cost =
mapper.evaluate_bitstring("".join(map(str,
x_test)), already_asset_order=True)
    # build matching Qiskit-order string
and evaluate via the raw (non-already-
asset-order) path
    qiskit_order_str = "".join(map(str,
x_test))[::-1]
    reconstructed_cost =
mapper.evaluate_bitstring(qiskit_order_str,
already_asset_order=False)
    print(f"Direct (asset order) cost:
```

```
{direct_cost:.6f}")
    print(f"Round-tripped (Qiskit order)
cost: {reconstructed_cost:.6f}")
    print(f"Match: {np.isclose(direct_cost,
reconstructed_cost)}")
```